My final report in assignment 3:

Here is my report on assignment 3. The code will be provided in this file as a reference on the last page.

1. Complete the table below and include it at the beginning of each required document listed on page 3 in the Submission section:

Unit Code		Assignment#	3
Student ID Number		Student Name	
Tutor's Name		Workshop Date/Time	

unit Code:6350

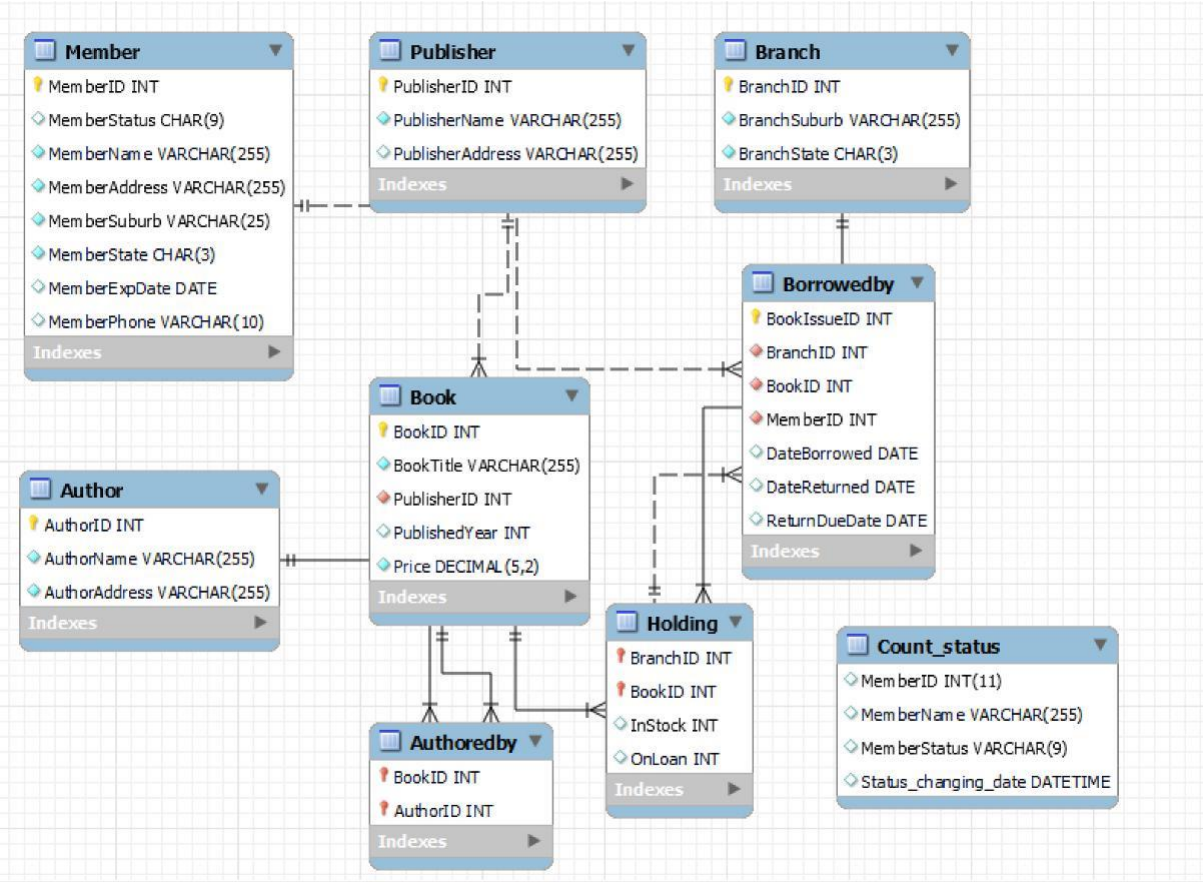
Student ID:45245673

STudent Name:Gary lau

WORKSHOP DATE/TIME : Thursday 6-8 pm

tutor:

Schema for the table:



Graph(A): our Schema

Key: (Part of) Primary Key
Filled Diamond: NOT NULL
Not filled Diamond: NULL
Red colored: (Part of) Foreign key
Blue lined Diamond: Simple attribute (no key)

Can be combined for example:
is a Red colored Key so it's a Primary Key which is also a Foreign Key
is a Yellow (non Red) Key so it's only a Primary Key
is a blue lined filled diamond so it's a NOT NULL simple attribute
is a red colored filled diamond so it's a NOT NULL Foreign Key
is a blue lined not filled diamond so it's a simple attribute which can be NULL
is a red colored not filled diamond so it's a Foreign Key which can be NULL

Graph (B) : this is the explanation which is from the internet
Photo from: https://stackoverflow.com/questions/10778561/what-do-the-mysql-workbench-column-icons-mean

Branch (BranchID [PK], BranchSuburb, BranchState)

Member (MemberID [PK],MemberStatus,MemberName,MemberAddress,MemberSuburb,
,MemberState,
MemberExpDate,
MemberPhone
)

Publisher (
PublisherID [PK],
PublisherName ,
PublisherAddress,

)

Book (
BookID [PK],
BookTitle,
PublisherID[fk],
PublishedYear,
Price

)

Author (
AuthorID [pk]
AuthorName,
AuthorAddress

)

Authoredby (
BookID[pk,fk],
AuthorID[pk,fk]

)

Holding (
  BranchID[pk,fk],
  BookID]pk,fk],
  InStock,
  OnLoan

)

TABLE Borrowedby (
  BookIssueID[pk],IssueID[pk],
  BranchID[fk],
  BookID[fk],
  MemberID[fk],
  DateBorrowed,
  DateReturned,
  ReturnDueDate
)

Report of Part 1(a):

In the first task, we created a table called fine logging to store all information about fines for each member.

```
# question1: insert a new column to record the fine in table member

ALTER TABLE Member
ADD COLUMN fine_logging DOUBLE(10,2);
```

Report of Part 1(b):

We will run a helper procedure(which we will define later) to calculate all fine fees for all later members.

```
174     VALUES ('26','REGULAR','peter_21','5 Nowhere St','Here','NSW','
175 •   INSERT INTO Member (MemberID,MemberStatus,MemberName,MemberAddr
176     VALUES ('27','REGULAR','peter_22','5 Nowhere St','Here','NSW','
177 •   INSERT INTO Member (MemberID,MemberStatus,MemberName,MemberAddr
178     VALUES ('28','REGULAR','peter_23','5 Nowhere St','Here','NSW','
179
```

Result Grid | Filter Rows: | Edit: | Export/Import:

MemberID	MemberStatus	fine_logging	MemberExpDate	MemberName	MemberAddres
1	REGULAR	NULL	2021-09-30	Joe	4 Nowhere St
2	REGULAR	NULL	2022-09-30	Pablo	10 Somewhere S
3	REGULAR	NULL	2020-11-30	Chen	23/9 Faraway C
4	REGULAR	NULL	2020-12-31	Zhang	Dunno St

We see that member ID 1,2 never returned book in bookID 1 ,2 and 4.

```
1 •   select * from borrowedby;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

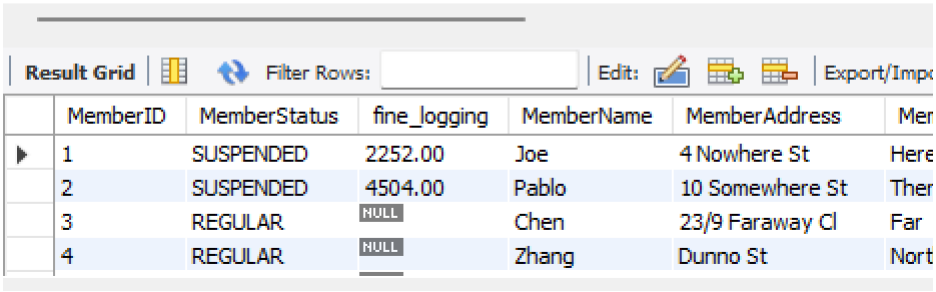
BookIssueID	BranchID	BookID	MemberID	DateBorrowed	DateReturned	ReturnDueDate
4	2	4	1	2023-10-31	NULL	2023-11-21
6	1	1	1	2020-08-30	NULL	2020-09-30
1	1	1	2	2023-10-31	NULL	2023-11-21
7	1	2	2	2020-08-30	NULL	2020-09-30
8	3	4	2	2020-08-30	NULL	2020-09-30

We now run an extra procedure to update the fine for someone according to the rule which is called overdue_countfine()

```
416  13:30:34  select * from borrowedby LIMIT 0, 1000   8 row(s) returned
417  13:32:15  call overdue_countfine()                  0 row(s) affected
```

Now it becomes suspended due to the trigger we define in Question(2):

```
4
5 •    select * from member;
```



Result Grid | Filter Rows: | Edit: | Export/Impo

MemberID	MemberStatus	fine_logging	MemberName	MemberAddress	Mer
1	SUSPENDED	2252.00	Joe	4 Nowhere St	Here
2	SUSPENDED	4504.00	Pablo	10 Somewhere St	Ther
3	REGULAR	NULL	Chen	23/9 Faraway Cl	Far
4	REGULAR	NULL	Zhang	Dunno St	Nort

We also see that the fines are being recorded as well.

e.g. Joe has a very high fine as he borrowed two books for three years (per day 2 dollars).

Part(2)(a):

```
drop trigger if exists overdue ;
CREATE trigger overdue
BEFORE UPDATE ON member
for each ROW
begin
DECLARE msg VARCHAR (255);
if NEW.fine_logging<0 then
set msg ="wrong output in the fine";
SIGNAL SQLSTATE "45000" SET MESSAGE_TEXT=msg;
#cannot get negative input in the fine
ELSE
IF(NEW.fine_logging=0 or NEW.fine_logging is NULL)and (OLD.MemberStatus= "SUSPENDED") then
SET NEW.memberStatus= "REGULAR";
ELSEIF ((NEW.fine_logging!=0 or NEW.fine_logging is NOT NULL) AND OLD.MemberStatus= "SUSPENDED"
if NEW.MemberStatus= "REGULAR" then
set msg ="suspended member didnt clear the fine or this is wrong input";
SIGNAL SQLSTATE "45000" SET MESSAGE_TEXT=msg;
END if;
#this is for error handling part since we cannot set the only who has suspended status
#into regular status
END if;
END IF ;
END //
DELIMITER ;
```

Here is our code in trigger.
We have two error handling which signal the errors.

(1) This doesn't seem right if we set a fine that is less than 0.
(2) We cannot turn SUSPENDED into REGULAR if the fine is not clear yet.

The way that I create this trigger is to build it up step by step. I consider if the fine is 0 or not 0 when the state is suspended for each member.

```
update member   #case one,we test whether the trigger works or not.
set fine_logging=0 where memberID=1;
```

Output

Action Output

#	Time	Action	Message
416	13:30:34	select * from borrowedby LIMIT 0, 1000	8 row(s) returned
417	13:32:15	call overdue_countfine()	0 row(s) affected
418	13:33:08	select * from member LIMIT 0, 1000	28 row(s) returned
419	13:44:46	select * from member LIMIT 0, 1000	28 row(s) returned
420	13:46:09	update member #case one,we test whether the trigger works or not. set fi...	1 row(s) affected Rows matched: 1 Changed: 1 Warni

Since Joe has cleared the fine,it turned out that:

```
837
838 •   select * from member;
839
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

MemberID	MemberStatus	MemberName	fine_logging	MemberAddress	MemberSuburb	MemberState	Mem
1	REGULAR	Joe	0.00	4 Nowhere St	Here	NSW	2021
2	SUSPENDED	Pablo	4504.00	10 Somewhere St	There	ACT	2022
3	REGULAR	Chen	NULL	23/9 Faraway Cl	Far	QLD	2020
4	REGULAR	Zhang	NULL	Dunno St	North	NSW	2020

member 3

Output

Action Output

#	Time	Action	Message
417	13:32:15	call overdue_countfine()	0 row(s) affected
418	13:33:08	select * from member LIMIT 0, 1000	28 row(s) returned
419	13:44:46	select * from member LIMIT 0, 1000	28 row(s) returned
420	13:46:09	update member #case one,we test whether the trigger works or not. set fi...	1 row(s) affected Rows matched:
421	13:46:55	select * from member LIMIT 0, 1000	28 row(s) returned

Now we can see Joe is a regular member.

Part (2b)

error handling:

We cannot set the (1) member status to be regular if he/she didn't clear the fine or (2) set the fine to be negative here. Here is our evidence to show there is an error signal when we set the one who is suspended into the regular state if he/she didn't clear the fine.

We see that Pablo hasn't cleared the fine yet. According to our rule it cannot be changed.

```
839
840 •   update member
841     set fine_logging=11 where memberID=2;
842
843 •   select * from member;
```

Result Grid | Filter Rows: | Edit: | Exp

MemberID	MemberStatus	fine_logging	MemberName	MemberAddress
1	REGULAR	0.00	Joe	4 Nowhere St
2	SUSPENDED	11.00	Pablo	10 Somewhere St
3	REGULAR	NULL	Chen	23/9 Faraway Cl
4	REGULAR	NULL	Zhang	Dunno St

Now if we change it:

```
# We cannot turn a suspened member into regular member if the fine is not cleared yet.
update member
set fine_logging=11 where memberID=2;
UPDATE member
SET memberstatus="REGULAR"
WHERE MemberID = 2;

select * from member;
#--------------We can see the fine become zero=> become regular again

update member
set fine_logging=122 where memberID=12;
update member
set fine_logging=0 where memberID=12;
```

ction Output

	Time	Action	Message
428	13:50:44	UPDATE member SET memberstatus="REGULAR" WHERE MemberID = 2	Error Code: 1644. suspended member didnt clear the fine o
429	13:50:44	UPDATE member SET memberstatus="REGULAR" WHERE MemberID = 2	Error Code: 1644. suspended member didnt clear the fine o
430	13:50:44	UPDATE member SET memberstatus="REGULAR" WHERE MemberID = 2	Error Code: 1644. suspended member didnt clear the fine o
431	13:50:44	UPDATE member SET memberstatus="REGULAR" WHERE MemberID = 2	Error Code: 1644. suspended member didnt clear the fine o
432	13:50:44	UPDATE member SET memberstatus="REGULAR" WHERE MemberID = 2	Error Code: 1644. suspended member didnt clear the fine o

This is blocked with a error message.

We also have the second error handle:

Let's say if we set the fine to be negative for this member:

```
846     #set the fine to become negative->error
847 •   UPDATE member
848     set fine_logging=-11 where memberID=2;
849
850 •   select * from member;
851     #--------------We can see the fine become zero=> become regular again
852
853 •   update member
854     set fine_logging=122 where memberID=12;
855 •   update member
856     set fine_logging=0 where memberID=12;
```

Output

Action Output

#	Time	Action	Message
435	13:52:39	UPDATE member set fine_logging=-11 where memberID=2	Error Code: 1644. wrong output in the fine
436	13:52:39	UPDATE member set fine_logging=-11 where memberID=2	Error Code: 1644. wrong output in the fine
437	13:52:40	UPDATE member set fine_logging=-11 where memberID=2	Error Code: 1644. wrong output in the fine
438	13:52:45	UPDATE member set fine_logging=-11 where memberID=2	Error Code: 1644. wrong output in the fine
439	13:52:45	UPDATE member set fine_logging=-11 where memberID=2	Error Code: 1644. wrong output in the fine

And this works again.

Report of Part 3(a)(b):

We add a table, a helper trigger, and a procedure to do it. I attempt to use a helper trigger to help keep track of new records in a new table when a member is from a regular to a suspended state. Then we can use a cursor in a procedure to display all results and update those records to terminate if they met the condition (e.g., within 3 years and suspended twice).

(1) We created a helper table called count_status and a helper trigger
Every time a member becomes suspended, it will automatically record in such need table count_status.

```
drop trigger if exists overdue_helper;
CREATE trigger overdue_helper
BEFORE UPDATE ON Member
FOR EACH ROW
begin
DECLARE msg VARCHAR (255);
if NEW.fine_logging<0 then
set msg ="wrong output in the fine";
SIGNAL SQLSTATE "45000" SET MESSAGE_TEXT=msg;
ELSE
IF NEW.fine_logging>=30 and OLD.MemberStatus= "REGULAR" then

INSERT INTO Count_status(MemberID,MemberStatus,Status_changing_date)
VALUES
(NEW.MemberID,"SUSPENDED",NOW());
END IF;

IF NEW.fine_logging>30 and OLD.MemberStatus= "REGULAR" then
SET NEW.memberStatus= "SUSPENDED";

END IF;
END IF;
END //
DELIMITER ;
```

This is our helper trigger above.

And We have our main procedure:

```
DELIMITER //
DROP PROCEDURE if exists overdue_procedure//
CREATE procedure overdue_procedure()
BEGIN
DECLARE v_MemberID INT;
DECLARE v_count INT;
DECLARE v_finish INT default 0;
DECLARE s_memberID INT;
DECLARE s_name VARCHAR(255);
DECLARE s_log Double(10,2);
declare overdue_handler CURSOR FOR
select count(MemberID),MemberID
from count_status
where NOW()< DATE_ADD(Status_changing_date, INTERVAL 3 YEAR)
group by MemberID
having count(MemberID)>1;

DECLARE CONTINUE HANDLER for not found set v_finish=1;
DECLARE CONTINUE HANDLER for sqlexception
BEGIN
ROLLBACK;
Select "fail to terminate";
END;
```

```
326     open overdue_handler;
327     repeat
328     fetch overdue_handler INTO v_count,v_MemberID;
329     SELECT MemberID,MemberName,fine_logging into s_memberID,s_name,s_log
330     from Member
331     where MemberID=v_MemberID;
332     if s_log>0 then
333     SELECT s_memberID,s_name;
334     ELSE
335     SIGNAL SQLSTATE "45000";
336     end if;
337     UPDATE member
338     SET MemberStatus="Terminate"
339     WHERE MemberID=v_MemberID and fine_logging>0;
340
341     UNTIL v_finish
342     END repeat;
343     CLOSE overdue_handler;
344     END//
345     DELIMITER ;
346
```

In this procedure we have error handling for exceptions.

If the fine of a member is zero even if that member has been suspended twice, this would not be displayed with a continue hander with an exception.

Member 1 and Member 2 has been suspended once due to the member expiration problem above, so we automatically have two records because of the trigger above:

```
 9
10 •   select * from count_status;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

MemberID	MemberName	MemberStatus	Status_changing_date
1	NULL	SUSPENDED	2023-10-31 13:32:15
2	NULL	SUSPENDED	2023-10-31 13:32:15
NULL	NULL	NULL	NULL

Now let's said Joe is suspended again because he damaged the book:

```
 7 •   update member
 8     set fine_logging=100 where memberID=1;
 9
10 •   select * from count_status;
```

Result Grid | Filter Rows: | Edit: | Export/Im

MemberID	MemberStatus	fine_logging	MemberName	MemberAddress	M
1	SUSPENDED	100.00	Joe	4 Nowhere St	He
2	SUSPENDED	11.00	Pablo	10 Somewhere St	Th
3	REGULAR	NULL	Chen	23/9 Faraway Cl	Fa
4	REGULAR	NULL	Zhang	Dunno St	No

Now we have two records:

```
879     #--------in question 3---we can display and update the termination using this code.
880     #set joe to be suspended twice
881
882     update member
883     set fine_logging=100 where memberID=1;
884
885     select * from count_status;
886
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

MemberID	MemberName	MemberStatus	Status_changing_date
1	NULL	SUSPENDED	2023-10-31 13:32:15
1	NULL	SUSPENDED	2023-10-31 14:02:08
2	NULL	SUSPENDED	2023-10-31 13:32:15
NULL	NULL	NULL	NULL

Also supposed we add two member who has been suspended twice:

For example, we have two member ID 26,25 who get into this list:

```
889     UPDATE member
890     SET fine_logging=0
891     WHERE MemberID = 26;
892
893     UPDATE member
894     SET fine_logging=111
895     WHERE MemberID = 26;
896
897     select * from member;
898
899     select * from count status;
```

Output

Action Output

#	Time	Action	Message
443	14:03:27	select * from count_status LIMIT 0, 1000	3 row(s) returned
444	14:04:21	call overdue_procedure()	1 row(s) returned
445	14:04:21	call overdue_procedure()	1 row(s) returned
446	14:04:59	UPDATE member SET fine_logging=1111 WHERE MemberID = 13	1 row(s) affected Rows matched: 1 Changed: 1 Warning
447	14:05:11	select * from member LIMIT 0, 1000	28 row(s) returned
448	14:05:40	UPDATE member SET fine_logging=1111 WHERE MemberID = 26	1 row(s) affected Rows matched: 1 Changed: 1 Warning
449	14:06:03	select * from member LIMIT 0, 1000	28 row(s) returned
450	14:06:18	UPDATE member SET fine_logging=0 WHERE MemberID = 26	1 row(s) affected Rows matched: 1 Changed: 1 Warning
451	14:06:19	UPDATE member SET fine_logging=111 WHERE MemberID = 26	1 row(s) affected Rows matched: 1 Changed: 1 Warning

We display our table:

```
909 •   select * from count_status;
910
911 •   call overdue_procedure();
912
913     #display the result
914
```

Result Grid | Filter Rows: | Edit: | Export/Import:

MemberID	MemberStatus	MemberName	fine_logging	MemberAddress	MemberS
1	SUSPENDED	Joe	100.00	4 Nowhere St	Here
2	SUSPENDED	Pablo	11.00	10 Somewhere St	There
25	REGULAR	peter_20	0.00	5 Nowhere St	Here
26	SUSPENDED	peter_21	111.00	5 Nowhere St	Here

We have four records as well:

```
909 •   select * from count_status;
910
911 •   call overdue_procedure();
912
913     #display the result
914
915 •   select MemberID,MemberStatus,fine_logging from Member;
916 •   select * from count status;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

MemberID	MemberName	MemberStatus	Status_changing_date
1	NULL	SUSPENDED	2023-10-31 13:32:15
1	NULL	SUSPENDED	2023-10-31 14:02:08
2	NULL	SUSPENDED	2023-10-31 13:32:15
13	NULL	SUSPENDED	2023-10-31 14:04:59
25	NULL	SUSPENDED	2023-10-31 14:07:54
25	NULL	SUSPENDED	2023-10-31 14:07:56
26	NULL	SUSPENDED	2023-10-31 14:05:40

We can see that member 1 and 25 will be terminated in such case.

After calling the procedure:

```
911 •   call overdue_procedure();
912
913     #display the result
914
915 •   select MemberID,MemberStatus,fine_logging from Member;
916 •   select * from count status;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	s_memberID	s_name
▶	1	Joe

Result 15 × | Result 16 | Result 17 | Result 18

Output

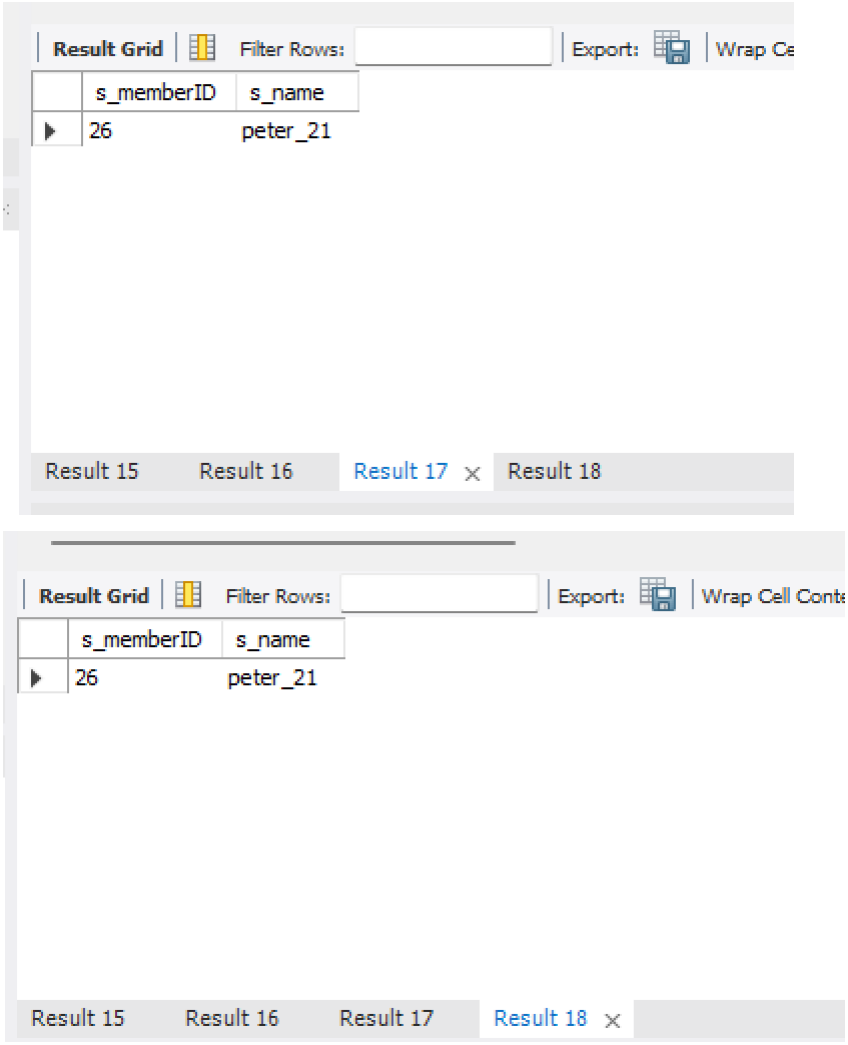
Action Output

	#	Time	Action	Message
✓	459	14:10:06	select * from count_status LIMIT 0, 1000	8 row(s) returned
✓	460	14:10:57	call overdue_procedure()	1 row(s) returned
✓	461	14:10:57	call overdue_procedure()	1 row(s) returned
✓	462	14:10:57	call overdue_procedure()	1 row(s) returned
✓	463	14:10:57	call overdue_procedure()	1 row(s) returned

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	fail to terminate
▶	fail to terminate

Result 15 | Result 16 × | Result 17 | Result 18



We display that all members (only 1 and 26) will become terminated but not 2 (#the suspended number is less than 2) or 25 (#the fine is zero, in this case, we have a message “fail to terminate” as expected).

Next, we have a final check:

```
907 •  select * from member where MemberID=25 or MemberID=26 or MemberID=1 or MemberID=2 ;
908
909 •  select * from count_status;
910
911 •  call overdue_procedure();
912
913    #display the result
914
915 •  select MemberID,MemberStatus,fine_logging from Member;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

MemberID	MemberStatus	MemberName	MemberAddress	MemberSuburb	MemberState	MemberExpDate	Membe
1	Terminate	Joe	4 Nowhere St	Here	NSW	2021-09-30	043456
2	SUSPENDED	Pablo	10 Somewhere St	There	ACT	2022-09-30	041234
25	REGULAR	peter_20	5 Nowhere St	Here	NSW	2024-09-30	043456
26	Terminate	peter_21	5 Nowhere St	Here	NSW	2024-09-30	043456
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Report of Part 4:

Our plan:

We need to test many cases we have. Due to the complicated nature of the task. We have created many triggers and procedures to make sure ALL OF THE rules fit. For example, we have set that only members with “REGULAR” member status can borrow books with a helper trigger, otherwise, it would be blocked. In this code, if a new member wants to borrow a book (before inserting a new row), we will see whether he is a regular member or not.

Here is a list of the triggers or procedure that I created, and I put the codes in the file:

(1) We have a trigger that updates the book into holding at the same time so that they will be in consistency.

```
12 •   select * from book;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

BookID	BookTitle	PublisherID	PublishedYear	Price
1	Anna Karenina	1	2003	12.75
2	War and Peace	2	1869	139.99
3	The Hobbit	2	1937	9.19
4	I, Robot	2	1950	29.99
5	The Positronic Man	3	2010	125.99
6	The Positronic Man	3	2010	125.99
NULL	NULL	NULL	NULL	NULL

We inserted many books at the same time:

```
653 •  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
654    VALUES ('7','New_test_book','1','2011',125.99);
655 •  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
656    VALUES ('8','The peterman_2','1','2012',125.99);
657 •  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
658    VALUES ('9','The peterman_3','2','2009',125.99);
659 •  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
660    VALUES ('10','The peterman_4','2','2009',125.99);
661 •  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
662    VALUES ('15','The peterman_6','2','2009',125.99);
663 •  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
664    VALUES ('16','The peterman_8','2','2009',125.99);
```

Output

Action Output

#	Time	Action	Message
472	14:29:05	INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price) ...	1 row(s) affected
473	14:29:05	INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price) ...	1 row(s) affected
474	14:29:05	INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price) ...	1 row(s) affected
475	14:29:05	INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price) ...	1 row(s) affected
476	14:29:05	INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price) ...	1 row(s) affected
477	14:29:05	INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price) ...	1 row(s) affected
478	14:29:05	INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price) ...	1 row(s) affected
479	14:29:05	INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price) ...	1 row(s) affected
480	14:29:05	INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price) ...	1 row(s) affected

Next, we can see from holding, that the books will be automatically updated:

```
689 •   select * from holding;
690 •   select * from borrowedby;
691 •   call borrow_holding(15,20,curdate(),NULL);
692 •   call borrow_holding(15,16,curdate(),NULL);
693 •   call borrow_holding(15,19,curdate(),NULL);
```

Result Grid | Filter Rows: | Edit:

BranchID	BookID	InStock	OnLoan
3	16	2	0
3	17	2	0
3	18	2	0
3	19	2	0
3	20	2	0
3	21	2	0
3	22	2	0
3	23	2	0
3	24	2	0

holding 1

(2)We created a procedure borrow holding where we add a member who borrowed a book and counts the INSTOCK and ONLOAD:

To keep track of the borrow-by and holding table. We created this procedure in which I can update the borrrowby and holding at the same time with members, BOOKID and date borrowed and date returned(can be NULL:

In this case, we call the procedure where the one who borrowed the book is member #15 And the 23 is the book we updated. And the return day is null.

✓	482	14:33:21	call borrow_holding(15,23,curdate(),NULL)	1 row(s) affected

```
1  select * from borrowedby where memberID=15;
2
3  call overdue_countfine();
4
5  select * from member;
6
7  update member
8  set fine_logging=100 where memberID=1;
9
10 select * from count_status;
11
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

BookIssueID	BranchID	BookID	MemberID	DateBorrowed	DateReturned	ReturnDueDate
9	3	23	15	2023-10-31	NULL	2023-11-21
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Now we have the record, and at the same time we have one record in holding:

```
688
689  select * from holding where BOOKID=23;
690  select * from borrowedby;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

BranchID	BookID	InStock	OnLoan
3	23	1	1
NULL	NULL	NULL	NULL

How about if he turned the book:

```
817
818  call borrow_holding(21,17,curdate(),curdate());
819  #return on time
699  select * from borrowedby where BOOKID=17;
700
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Co

BookIssueID	BranchID	BookID	MemberID	DateBorrowed	DateReturned	ReturnDueDate
10	3	17	21	2023-10-31	2023-10-31	2023-11-21
NULL	NULL	NULL	NULL	NULL	NULL	NULL

And the record is being added:

```
698
699 •  select * from holding where BOOKID=17;
700
```

Result Grid | Filter Rows: | Edit: | Export/Impor

BranchID	BookID	InStock	OnLoan
3	17	2	0
NULL	NULL	NULL	NULL

We can see that the holding has the consistent inStock and Onloan.

We also have a error signal if we return the same book again since ONLoan cannot be negative:

```
695 •  call borrow_holding(21,17,curdate(),curdate());
696 •  call borrow_holding(21,20,curdate(),NULL);
697
698
```

Output

Action Output

#	Time	Action	Message
511	14:59:52	call borrow_holding(21,17,curdate(),curdate())	Error Code: 1644. error: on_loan cannot be negative
512	14:59:53	call borrow_holding(21,17,curdate(),curdate())	Error Code: 1644. error: on_loan cannot be negative
513	14:59:53	call borrow_holding(21,17,curdate(),curdate())	Error Code: 1644. error: on_loan cannot be negative

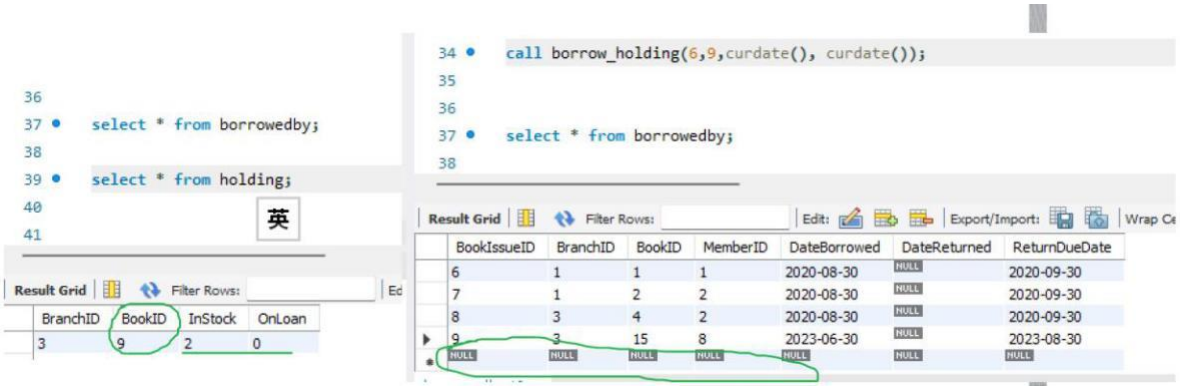
Other example:

```
DELIMITER //
DROP PROCEDURE if exists borrow_holding//
CREATE procedure borrow_holding(in mem_id INT,in BOOK_Id INT,in Date_borr DATE,in date_return
BEGIN
if date_return is NULL then
INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,DateReturned,ReturnDueDate)
VALUES ('3', BOOK_Id,mem_id,Date_borr,date_return,date_add(Date_borr,INTERVAL 3 WEEK));
END if;

if date_return is NOT NULL then
UPDATE Borrowedby
SET DateReturned = date_return
WHERE BranchID=3 and BOOKID=BOOK_Id and DateBorrowed=Date_borr;
END IF;

if date_return is NULL then
UPDATE Holding
set OnLoan=OnLoan+1 where BookID=BOOK_Id and InStock>1;
end if;
if date_return is NULL then
UPDATE Holding
set InStock=InStock-1 where BookID=BOOK_Id and InStock>1;
end if;
if date_return is NOT NULL then
UPDATE Holding
```

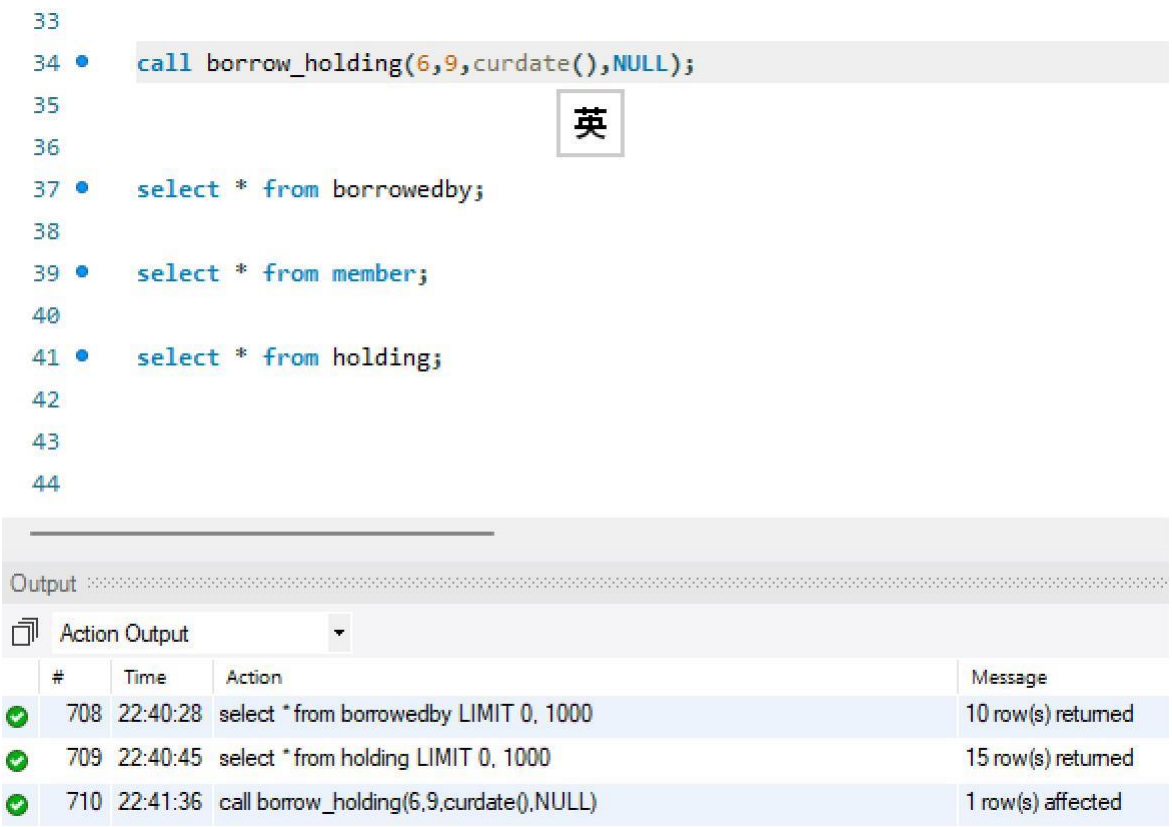
```
20      end if;
21      if date_return is NULL then
22      UPDATE Holding
23      set InStock=InStock-1 where BookID=BOOK_Id and InStock>1;
24      end if;
25      if date_return is NOT NULL then
26      UPDATE Holding
27      set OnLoan=OnLoan-1 where BookID=BOOK_Id;
28      end if;
29      if date_return is NOT NULL then
30      UPDATE Holding
31      set InStock=InStock+1 where BookID=BOOK_Id;
32      end if;
33      END//
34      DELIMITER ;
```

Case (1): When a member borrowed a book in borrow by and the book is not returned, using the procedure.

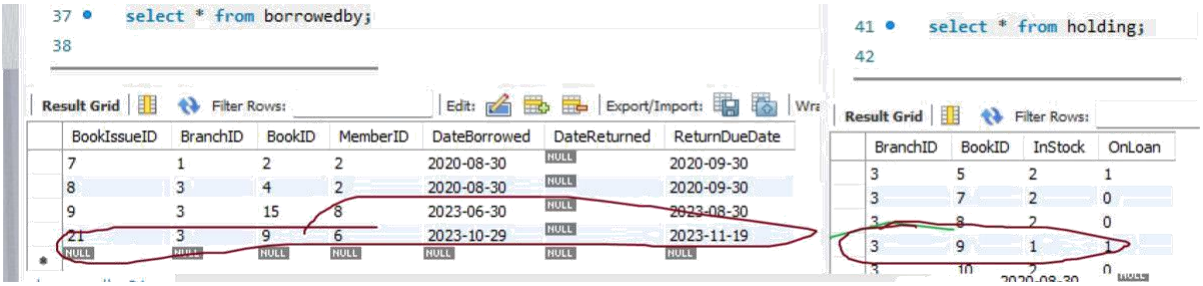
Step1: Before the procedure



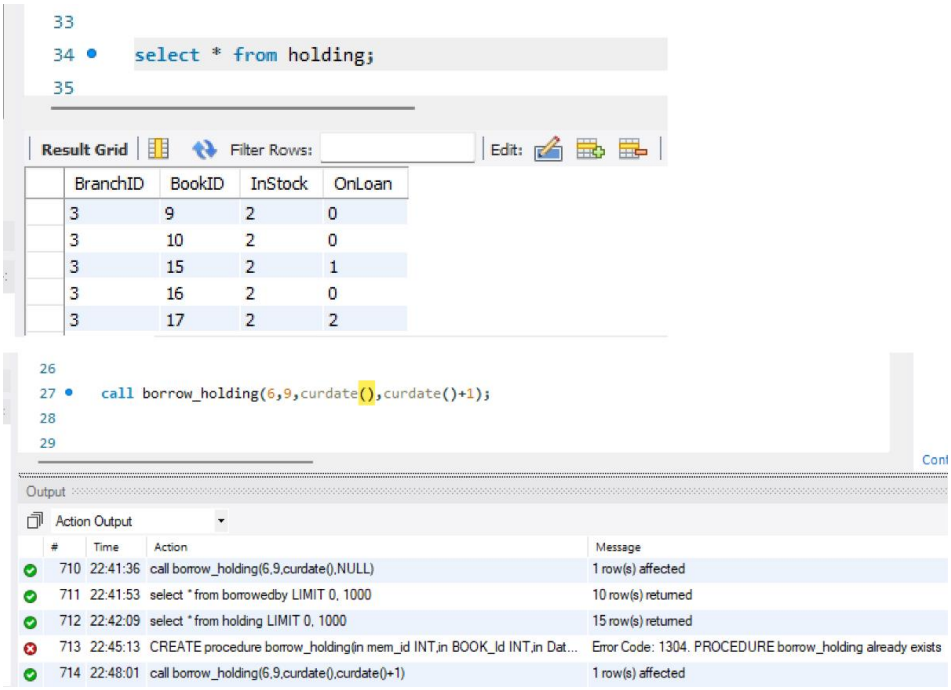
Step(2): apply the procedure and update a member who borrowed a book(in the example the member iD 8 borrowed a bookID 9 and we call the procedure.



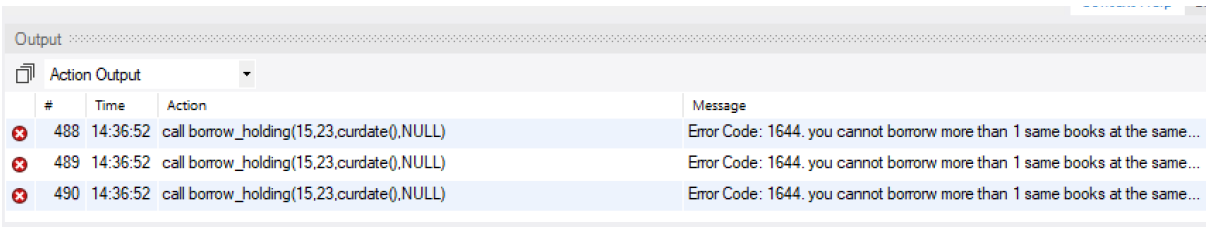
Step (3): We can see that the INSTOCK become 1 and ONLOAD +1 as well.



Step(4): the member returned the book so I call the procedure and update the table again.



(3) We also have triggers that limit the person who can borrow the same books within the same day. (BR3. Each member can borrow one copy of the same book on the same day.) We have already seen member 14 borrow a book#23 in the previous example. If we run the code and borrow the same book today again, it will fail.



(4) We also have other triggers that make limitations where a person can borrow five books within 3 weeks. (BR4. A member can borrow up to 5 items for 3 weeks (i.e., 21 days).)

let’s suppose a person# 21 who wants to borrow five books within three weeks.

```
691 •   call borrow_holding(21,22,curdate(),NULL);
692 •   call borrow_holding(21,21,curdate(),NULL);
693 •   call borrow_holding(21,19,curdate(),NULL);
694 •   call borrow_holding(21,18,curdate(),NULL);
695 •   call borrow_holding(21,17,curdate(),NULL);
696 •   call borrow_holding(21,20,curdate(),NULL);
697
```

Result Grid | Filter Rows: | Edit: | Expor

BranchID	BookID	InStock	OnLoan
2	1	1	1
3	17	1	1
3	18	1	1
3	19	1	1
3	21	1	1
3	22	1	1

10	3	17	21	2023-10-31	NULL	2023-11-21
11	3	22	21	2023-10-31	NULL	2023-11-21
12	3	21	21	2023-10-31	NULL	2023-11-21
13	3	19	21	2023-10-31	NULL	2023-11-21
14	3	18	21	2023-10-31	NULL	2023-11-21
NULL	NULL	NULL	NULL	NULL	NULL	NULL

If he wants to borrow the other book, it will turn out to be wrong:

```
call borrow_holding(21,16,curdate(),NULL);


call borrow_holding(15,23,curdate(),NULL);


select memberID,count(memberID) from borrowedby natural join holding
WHERE dateborrowed > date_sub(now(), interval 3 week) and memberID=15
group by memberID;



#We can use other tiggers to test whether we violate the business rule above.
```

Output

Action Output

#	Time	Action	Message
501	14:42:49	call borrow_holding(21,16,curdate(),NULL)	Error Code: 1644. you cannot borrorw more than 5 book
502	14:42:49	call borrow_holding(21,16,curdate(),NULL)	Error Code: 1644. you cannot borrorw more than 5 book
503	14:42:49	call borrow_holding(21,16,curdate(),NULL)	Error Code: 1644. you cannot borrorw more than 5 book

(4) BR2. Only members with “REGULAR” member status can borrow books.

We make a trigger to prevent people from borrowing new books(from preventing inserting a new value into the table borrowedby) with the condition (1) This member borrows a book later than the expiration date or (2) this member is not in a regular state.

830	21:13:19	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:this member is not in regular state or the membersh...	0
831	21:13:24	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:this member is not in regular state or the membersh...	0

This works if we choose a regular member which is not suspended:

832 21:13:26 select * from member LIMIT 0, 1000 10 row(s) returned

```
INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,DateReturned,ReturnDueDate)
VALUES ('3', '4','10','2023-08-30',NULL,'2023-09-30');
```

But we cannot choose a member who is suspended:

MemberID	MemberStatus	MemberName	MemberAddress	MemberSuburb	MemberState	MemberExpDate	Member
1	SUSPENDED	Joe	4 Nowhere St	Here	NSW	2021-09-30	0434567

We cannot add member ID 1 into the table since the due date passes the memberexpdate.

```
31
32 •  INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,DateReturned,ReturnDueDate)
33     VALUES ('3', '4','1','2023-08-30',NULL,'2023-09-30');
34
35
36 •  update member
37     set MemberExpdate="2014-01-30" where memberID=10;
38
```

Output

Action Output

#	Time	Action	Message
839	21:20:25	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:this member is not in reg
840	21:20:31	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:this member is not in reg
841	21:20:32	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:this member is not in reg
842	21:20:32	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:this member is not in reg

(5)

BR1. Any book can be borrowed if and only if at least one copy of the book is being held at the branch.

We can make a trigger to keep BR1 consistent:

Suppose we have a book that is not in stock and the BOOK ID is 7:

Result Grid | Filter Rows: | Edit:

BranchID	BookID	InStock	OnLoan
3	4	4	0
3	5	2	1
3	6	1	0
3	7	0	0
NULL	NULL	NULL	NULL

holding 14 ×

Output

if we borrow book 7 from a regular member 10 it would be a failure:

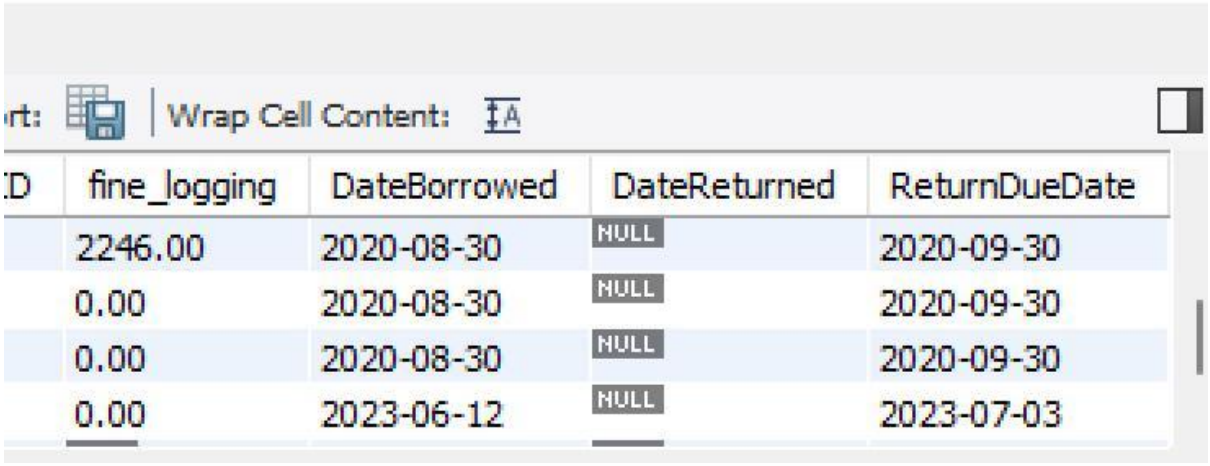
```
•  INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,DateReturned,ReturnDueDate)
   VALUES ('3', '7','10','2023-08-30',NULL,'2023-09-30');
```

#	Time	Action	Message
1035	22:08:11	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. Error: cannot borrow because it is not instock
1036	22:08:12	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. Error: cannot borrow because it is not instock

It will be blocked if we have a book that is not in stock.

(6) To keep the rule in BR7 consistent(+2 per day), we create a stored procedure called overdue_countfine() which can update all fine logging which including only overdue.
We have already seen an example in question 1b.

This is also easy to update the fine fee for the overdue book (due to simplicity).



ID	fine_logging	DateBorrowed	DateReturned	ReturnDueDate
	2246.00	2020-08-30	NULL	2020-09-30
	0.00	2020-08-30	NULL	2020-09-30
	0.00	2020-08-30	NULL	2020-09-30
	0.00	2023-06-12	NULL	2023-07-03

(7) for BR 8 it is already done in part (2).

(8)

BR5. The return due date of the borrowed book cannot be past the membership expiry date.

```
DELIMITER //
CREATE trigger businessrule_7
BEFORE insert ON borrowedby
FOR EACH ROW
begin
DECLARE v_count INT;
declare msg TEXT;
declare v_id INT;

declare v_exp DATE;

select memberID,MemberExpDate into v_id,v_exp from Member where
memberID = NEW.MemberID;

if (NEW.ReturnDueDate > v_exp) then
set msg ="wrong:the returneDuedate is later than expirationDate";
SIGNAL SQLSTATE "45000" SET MESSAGE_TEXT=msg;
END IF;

END //
DELIMITER ;
```

We can make a trigger like this

```
SELECT * from
member where MemberID=5;
```

Result Grid | Filter Rows: | Edit:

MemberID	MemberStatus	MemberExpDate	MemberName
5	REGULAR	2020-09-30	Saleem
NULL	NULL	NULL	NULL

We can see that member ID =5 has a expiration in 2020.

if #5 borrow a book and the return day is current time and it would be blocked.

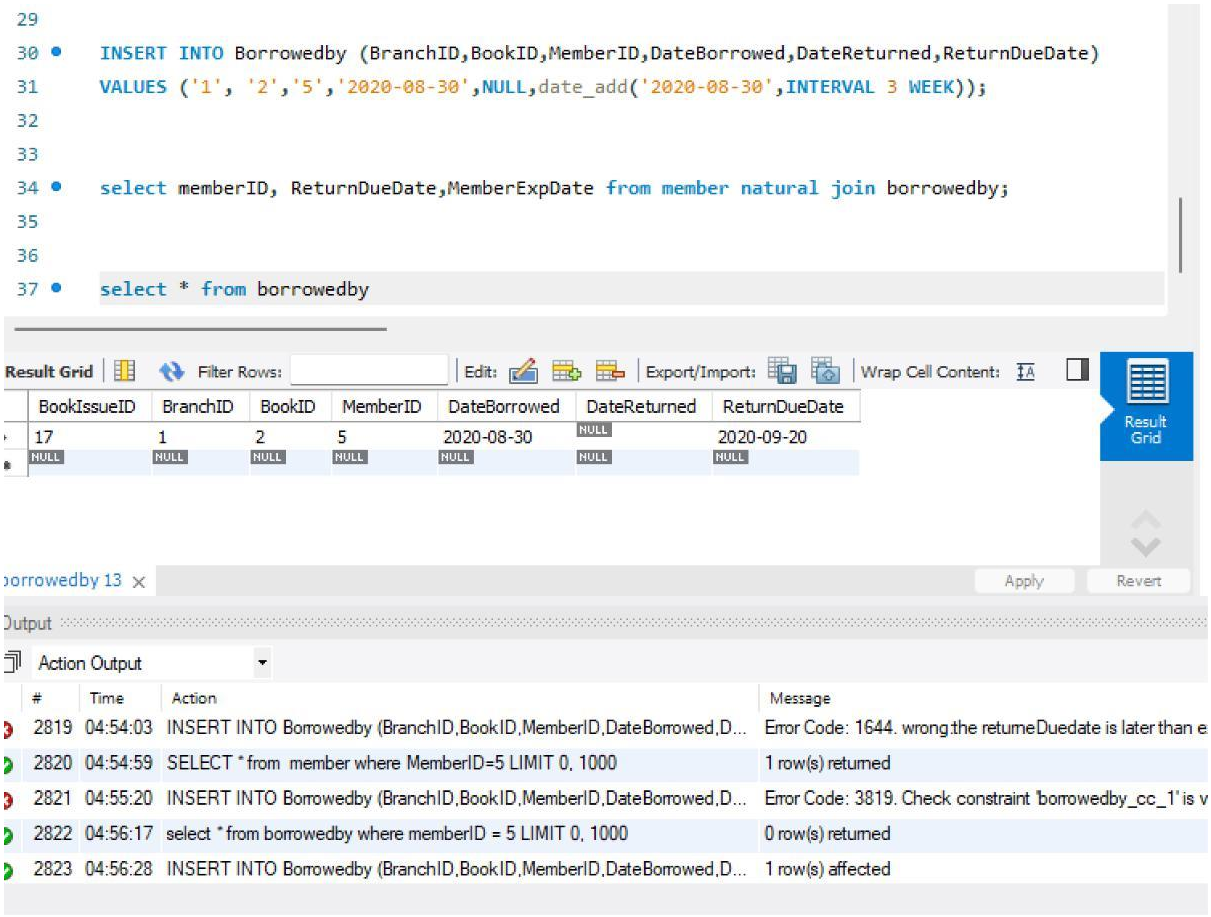
```
INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,DateReturned,ReturnDueDate)
VALUES ('1', '2','5',curdate(),NULL,date_add(curdate(),INTERVAL 3 WEEK));


select memberID, ReturnDueDate,MemberExpDate from member natural join borrowedby;


select * from Member
where
memberID = 2;
```

Output

Action Output

#	Time	Action	Message
2814	04:53:51	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:the returneDuedate is later tha
2815	04:54:01	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:the returneDuedate is later tha
2816	04:54:02	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:the returneDuedate is later tha
2817	04:54:02	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:the returneDuedate is later tha
2818	04:54:02	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:the returneDuedate is later tha

however, #5 can borrowed if the return due date is equal or earlier than that.



```
29
30 •  INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,DateReturned,ReturnDueDate)
31    VALUES ('1', '2','5','2020-08-30',NULL,date_add('2020-08-30',INTERVAL 3 WEEK));
32
33
34 •  select memberID, ReturnDueDate,MemberExpDate from member natural join borrowedby;
35
36
37 •  select * from borrowedby
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

BookIssueID	BranchID	BookID	MemberID	DateBorrowed	DateReturned	ReturnDueDate
17	1	2	5	2020-08-30	NULL	2020-09-20
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid

borrowedby 13 ×

Apply | Revert

Output

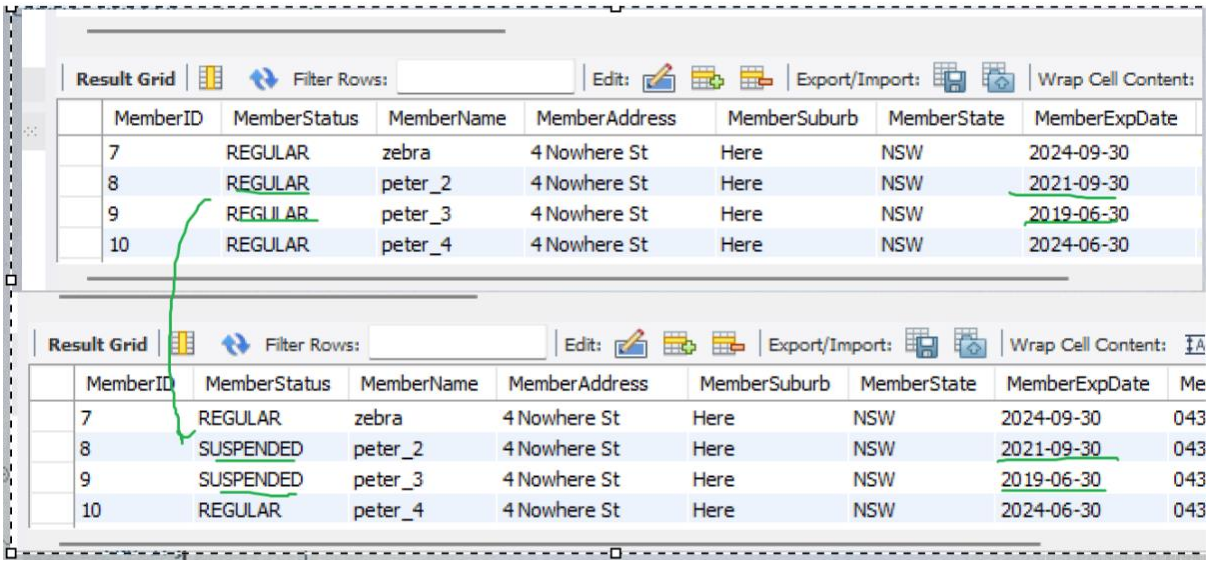
Action Output

#	Time	Action	Message
2819	04:54:03	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:the returneDuedate is later than e
2820	04:54:59	SELECT * from member where MemberID=5 LIMIT 0, 1000	1 row(s) returned
2821	04:55:20	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 3819. Check constraint 'borrowedby_cc_1' is v
2822	04:56:17	select * from borrowedby where memberID = 5 LIMIT 0, 1000	0 row(s) returned
2823	04:56:28	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	1 row(s) affected

(9): Convert all regular members into suspended members with a procedure.

We also need one procedure to update a member passes the expiration date.

```
559  DROP PROCEDURE if exists keep_suspend//
560  CREATE procedure keep_suspend()
561  BEGIN
562  DECLARE v_MemberID INT;
563  DECLARE v_status TEXT;
564  DECLARE v_Exp DATE ;
565  DECLARE v_finish INT default 0;
566
567  declare suspend_handler CURSOR FOR
568  select MemberID,MemberStatus,MemberExpDate
569  from Member;
570  DECLARE CONTINUE HANDLER for not found set v_finish=1;
571  open suspend_handler;
572  repeat
573  fetch suspend_handler INTO v_MemberID,v_status,v_Exp;
574  UPDATE member
575  SET MemberStatus="SUSPENDED"
576  WHERE MemberID=v_MemberID and MemberStatus="REGULAR" and (curdate()>v_Exp);
577  UNTIL v_finish
578  END repeat;
```



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

MemberID	MemberStatus	MemberName	MemberAddress	MemberSuburb	MemberState	MemberExpDate
7	REGULAR	zebra	4 Nowhere St	Here	NSW	2024-09-30
8	REGULAR	peter_2	4 Nowhere St	Here	NSW	2021-09-30
9	REGULAR	peter_3	4 Nowhere St	Here	NSW	2019-06-30
10	REGULAR	peter_4	4 Nowhere St	Here	NSW	2024-06-30

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

MemberID	MemberStatus	MemberName	MemberAddress	MemberSuburb	MemberState	MemberExpDate	Me
7	REGULAR	zebra	4 Nowhere St	Here	NSW	2024-09-30	043
8	SUSPENDED	peter_2	4 Nowhere St	Here	NSW	2021-09-30	043
9	SUSPENDED	peter_3	4 Nowhere St	Here	NSW	2019-06-30	043
10	REGULAR	peter_4	4 Nowhere St	Here	NSW	2024-06-30	043

This is how it works. When the membership of that person expires, it will turn into state SUSPENDED if it is originally in a REGULAR state.

At the same time, it would be recorded in count status as well.

(10):

BR2. Only members with “REGULAR” member status can borrow books.

We make a trigger to prevent people from borrowing new books(from preventing inserting a new value into the table borrowedby) with the condition (1) This member borrows a book later than the expiration date or (2) this member is not in a regular state.

#we have already terminated member 1 in the previous example, so he couldn’t borrow the book.

```
697
698 •  call borrow_holding(1,19,curdate(),NULL);
```

Context Help

Output

Action Output

#	Time	Action	Message
523	15:14:08	call borrow_holding(1,19,curdate(),NULL)	Error Code: 1644. wrong:this member is not in regular state or the membersh...
524	15:14:09	call borrow_holding(1,19,curdate(),NULL)	Error Code: 1644. wrong:this member is not in regular state or the membersh...
525	15:14:09	call borrow_holding(1,19,curdate(),NULL)	Error Code: 1644. wrong:this member is not in regular state or the membersh...

This works if we choose a member who is regular which is not suspended:

832 21:13:26 select * from member LIMIT 0, 1000 10 row(s) returned

```
INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,DateReturned,ReturnDueDate)
VALUES ('3', '4','10','2023-08-30',NULL,'2023-09-30');
```

Task (2): Test

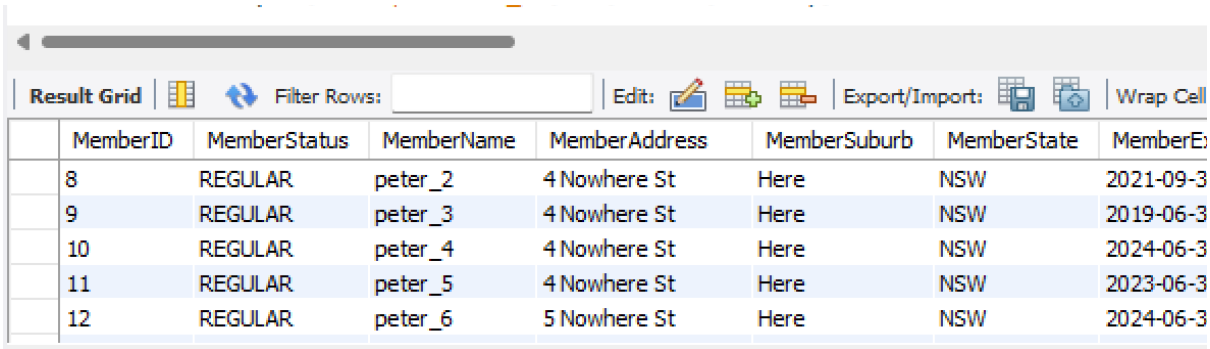
test plan:

the data we need:

(1) we need a person who has a fine which is suspended and the find>0 but it turned into zero fine later(to see whether the status will become regular)
(2) we need a person who has a fine that is suspended and the fine>0 but it turned into a non-zero fine later and to verify the state will not be changed. (the error handler will be activated if you change it to regular status)
(3) we need a terminated person and if we set the fine=0. This makes sure not to change the state incorrectly.
(4) we need the base case: the one that is regular with fine>0. And we change the fine into zero to see it doesn't have strange thing happens
(5) we need the base case: the one that is regular with fine=0. And we change the fine into non-zero to see it doesn't have strange thing happens

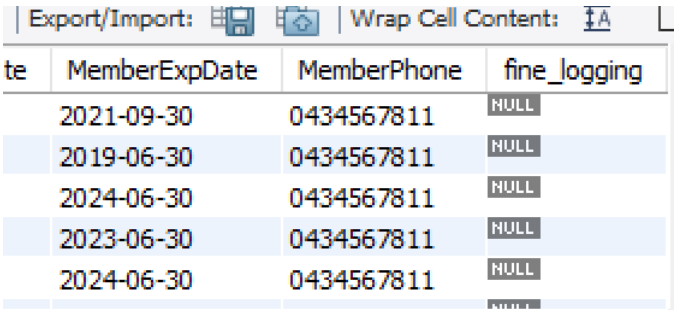
So in our SQL setup, we need to have those data inside the procedure.

After setting everything, we make sure that we haven't done anything wrong and this is the pre-planned stage. So we can add as much as we want. We just need to make sure that if a member borrowed a book, then I need to add on loan +1 in the book to prevent inconsistency since I don’ t have any trigger or procedure for this part due to the complexity.



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

MemberID	MemberStatus	MemberName	MemberAddress	MemberSuburb	MemberState	MemberEx
8	REGULAR	peter_2	4 Nowhere St	Here	NSW	2021-09-3
9	REGULAR	peter_3	4 Nowhere St	Here	NSW	2019-06-3
10	REGULAR	peter_4	4 Nowhere St	Here	NSW	2024-06-3
11	REGULAR	peter_5	4 Nowhere St	Here	NSW	2023-06-3
12	REGULAR	peter_6	5 Nowhere St	Here	NSW	2024-06-3



Export/Import: | Wrap Cell Content:

te	MemberExpDate	MemberPhone	fine_logging
	2021-09-30	0434567811	NULL
	2019-06-30	0434567811	NULL
	2024-06-30	0434567811	NULL
	2023-06-30	0434567811	NULL
	2024-06-30	0434567811	NULL

Next, we can put everything into the test case now. Since we have triggers to do all kinds of things, this part would be easier as we know many rules cannot be violated due to our design of the triggers. We just give some data to test it step by step to make sure it works.

Also, we add more books here:

```
484  select * from member;
485
486  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
487  VALUES ('7','New_test_book','1','2011',125.99);
488  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
489  VALUES ('8','The peterman_2','1','2012',125.99);
490  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
491  VALUES ('9','The peterman_3','2','2009',125.99);
492  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
493  VALUES ('10','The peterman_4','2','2009',125.99);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Co

BookID	BookTitle	PublisherID	PublishedYear	Price
3	The Hobbit	2	1937	9.19
4	I, Robot	2	1950	29.99
5	The Positronic Man	3	2010	125.99
6	The Positronic Man	3	2010	125.99
NULL	NULL	NULL	NULL	NULL

Since we have triggered,all books will automatically come to the table “HOLDING”..

```
490  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
491  VALUES ('9','The peterman_3','2','2009',125.99);
492  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
493  VALUES ('10','The peterman_4','2','2009',125.99);
494  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
495  VALUES ('15','The peterman_6','2','2009',125.99);
496  INSERT INTO Book (BookID,BookTitle,PublisherID,PublishedYear,Price )
497  VALUES ('16','The peterman_8','2','2009',125.99);
498
499  select * from holding;
500
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

BranchID	BookID	InStock	OnLoan
3	5	2	1
3	7	1	0
3	8	1	0
3	9	1	0
3	10	1	0

holding 38

Now we should check if we have a book first (in this example the ID # is 7):

	6	Petermum	2	1941	19.19
	7	New_test_book	1	1941	19.19
*	NULL	NULL	NULL	NULL	NULL

	3	6	1	0
	3	7	1	0
*	NULL	NULL	NULL	NULL

Also we can see there is one on the table and zero on loan.

Now if Peter_2 wants to borrow a book but this cannot work since his membership due late is earlier than 10/29/2023(now) .As a result our trigger will block this order.

```
504 •  INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,DateReturned,ReturnDueDate)
505    VALUES ('3', '7','8','2023-06-30',NULL,'2023-08-30');
506
507 •  select * from Borrowedby ;
508
```

Output

Action Output

#	Time	Action	Message
1828	00:38:48	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:this member is not in regular stat
1829	00:38:49	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:this member is not in regular stat
1830	00:38:50	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:this member is not in regular stat
1831	00:38:50	INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,D...	Error Code: 1644. wrong:this member is not in regular stat

We need to change the member expiration date of this member # 8(peter_2):

#	Time	Action	Message
1846	00:42:21	UPDATE member set MemberExpDate="2025-01-01" where memberID=8	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0

do it again:

```
INSERT INTO Borrowedby (BranchID,BookID,MemberID,DateBorrowed,DateReturned,ReturnDueDate)
VALUES ('3', '7','8','2023-06-30',NULL,'2023-08-30');
```

We can see this is true now:

```
511 •  select * from Borrowedby ;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

BookIssueID	BranchID	BookID	MemberID	DateBorrowed	DateReturned	ReturnDueDate
9	3	7	8	2023-06-30	NULL	2023-08-30

but we cannot add the same book again since this violates BR2.

Next We assume that peter has too much fine due to book damage/never return the books

We can use another helper_procedure to calculate the fine(which is a procedure to update the fine with 2* the data (1)between the return due day and the current day(if he returns the book) or (2) between the current day and the return due day (if he didn't return the books)

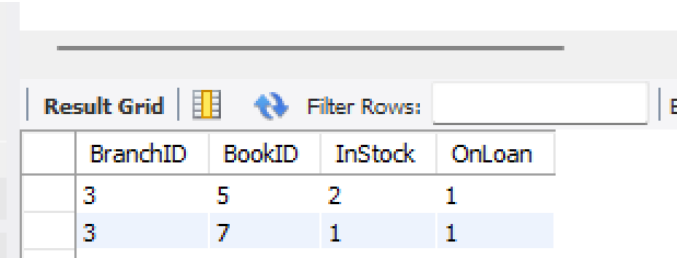
We also add a helper function to caluclate this within the procedure:

```
DROP function if exists count_fine;
CREATE function count_fine(Returned DATE,Return_due DATE)
RETURNS int
DETERMINISTIC
BEGIN
DECLARE count INT;
IF (Returned IS NULL) and DATEDIFF(curdate(), Return_due)>0 then
SET count =2*DATEDIFF(curdate(), Return_due);
end if;

IF (Returned IS NOT NULL) and DATEDIFF(Returned,Return_due)>0 then
SET count =2*DATEDIFF(Returned,Return_due);
end if;

return count;
end //
DELIMITER ;
```

We can see Peter_2 is in a state of suspension because he has too much fine which is greater than 30. (with 234.00)

8	peter_2	234.00	SUSPENDED
9	peter_3	NULL	REGULAR
10	peter_4	NULL	REGULAR

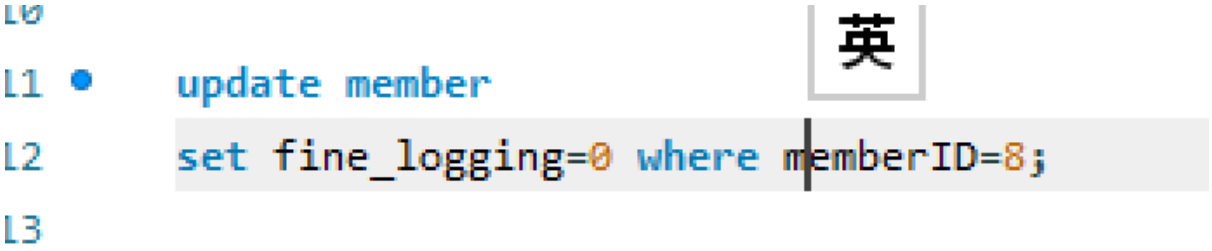
Also now one more book is on_loan. So we add 1 into OnLoan.



Result Grid | Filter Rows:

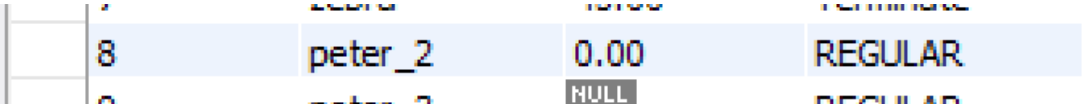
BranchID	BookID	InStock	OnLoan
3	5	2	1
3	7	1	1

Let's assume peter returned the book:



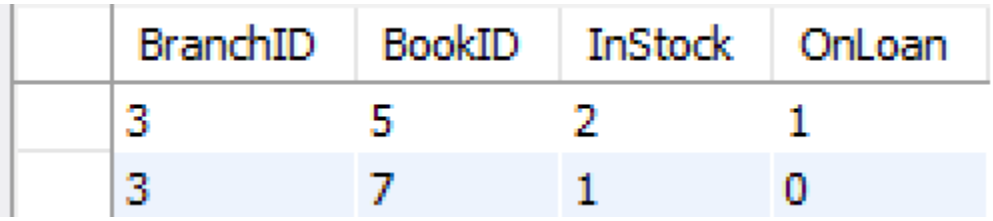
```
update member
set fine_logging=0 where memberID=8;
```

We can see now Peter has a regular fine and this proves that the trigger works in task 2.



8	peter_2	0.00	REGULAR
		NULL	

Since Peter returned the book, to keep consistency we need to reset the current holding where the BOOKID has 0 on loan.



BranchID	BookID	InStock	OnLoan
3	5	2	1
3	7	1	0

Case (1): We have already tested.

Case (3): We have already terminated member ID1, so we cannot set it to regular status even if the fine is 0.

```
907
908
909     # test -case (3) we cannot change a terminate status into regular status
910
911 •   select * from member
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

MemberID	MemberStatus	fine_logging	MemberName	MemberAddress	MemberSuburb	MemberState	MemberExpDat
1	Terminate	0.00	Joe	4 Nowhere St	Here	NSW	2021-09-30
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Case(2):

```
918     # test -case (2) we cannot change a suspened member into re
919
920 •   select * from member where memberID=19;
921
922 •   UPDATE member
923     SET fine_logging=100
924     WHERE MemberID = 19;
925
```

Result Grid | Filter Rows: | Edit: | Export/Import:

MemberID	MemberStatus	fine_logging	MemberName	MemberAddress	MemberSu
19	SUSPENDED	100.00	peter_14	5 Nowhere St	Here
NULL	NULL	NULL	NULL	NULL	NULL

If we try to change the fine for memebr19:

```
921
922 •   UPDATE member
923     SET fine_logging=1
924     WHERE MemberID = 19;
925
```

Result Grid | Filter Rows: | Edit: | Export/Import:

MemberID	MemberStatus	fine_logging	MemberName	MemberAddress	MemberSuburb	Me
19	SUSPENDED	1.00	peter_14	5 Nowhere St	Here	NS
NULL	NULL	NULL	NULL	NULL	NULL	NUL

member 24

Output

Action Output

#	Time	Action	Message
537	15:32:56	select * from member where memberID=19 LIMIT 0, 1000	1 row(s) ret
538	15:33:26	UPDATE member SET fine_logging=1 WHERE MemberID = 19	1 row(s) aff
539	15:33:29	select * from member where memberID=19 LIMIT 0, 1000	1 row(s) ret

It does nothing.

If we set him into regular status, it would fail:

```
917
918     # test -case (2) we cannot change a suspened member into regular member without clearing the fi
919
920 •   select * from member where memberID=19;
921
922 •   UPDATE member
923     SET MemBErstatus="REGULAR"
924     WHERE MemberID = 19;
925
926
```

Context Help | S

Output

Action Output

#	Time	Action	Message
541	15:34:44	UPDATE member SET MemBErstatus="REGULAR" WHERE MemberID = ...	Error Code: 1644. suspended member didnt clear the fine or this is wrong in...
542	15:34:44	UPDATE member SET MemBErstatus="REGULAR" WHERE MemberID = ...	Error Code: 1644. suspended member didnt clear the fine or this is wrong in...
543	15:34:44	UPDATE member SET MemBErstatus="REGULAR" WHERE MemberID = ...	Error Code: 1644. suspended member didnt clear the fine or this is wrong in...

(4) and (5) are easier to test since the trigger does not affect both cases.

If we set the REGULAR member to have a fine with 11. The state doesn't change.

```
777
778 •   select * from member
779     where MemberID=11;
```

Result Grid | Filter Rows: | Edit: | Export/Import

MemberID	MemberStatus	fine_logging	MemberName	MemberAddress	Member
11	REGULAR	11.00	peter_5	4 Nowhere St	Here
NULL	NULL	NULL	NULL	NULL	NULL

(5) is just a base case where it cannot be changed.

```
780
781 •   UPDATE member
782     SET fine_logging=0
783     WHERE MemberID = 11;
784
785
786
787 •   select * from member
788     where MemberID=11;
```

Result Grid | Filter Rows: | Edit:

MemberID	MemberStatus	fine_logging	MemberName	Membe
11	REGULAR	0.00	peter_5	4 Nowhe
NULL	NULL	NULL	NULL	NULL

Test (3):

test plan:

the data we need:

(1)we have a person who has been suspended twice or more within three years with a current fine!=0

(2)we need a person who has been suspended twice or more within three years with current fine=0

(3)we need a person who has been suspended twice or more but who is not within three year

(4)we need some data that has been suspended less than twice.

So in our SQL setup, we need to have those data inside the procedure.

Also, we have done (1) and (2) and (4) in the example in task (2) we have shown. But we will do one more example here.

Part(A): normal case (1)(2)(4)

Lets say Peter_3(member ID #8) has 2 suspended because of a late membership fee before and now he returned a damaged book.

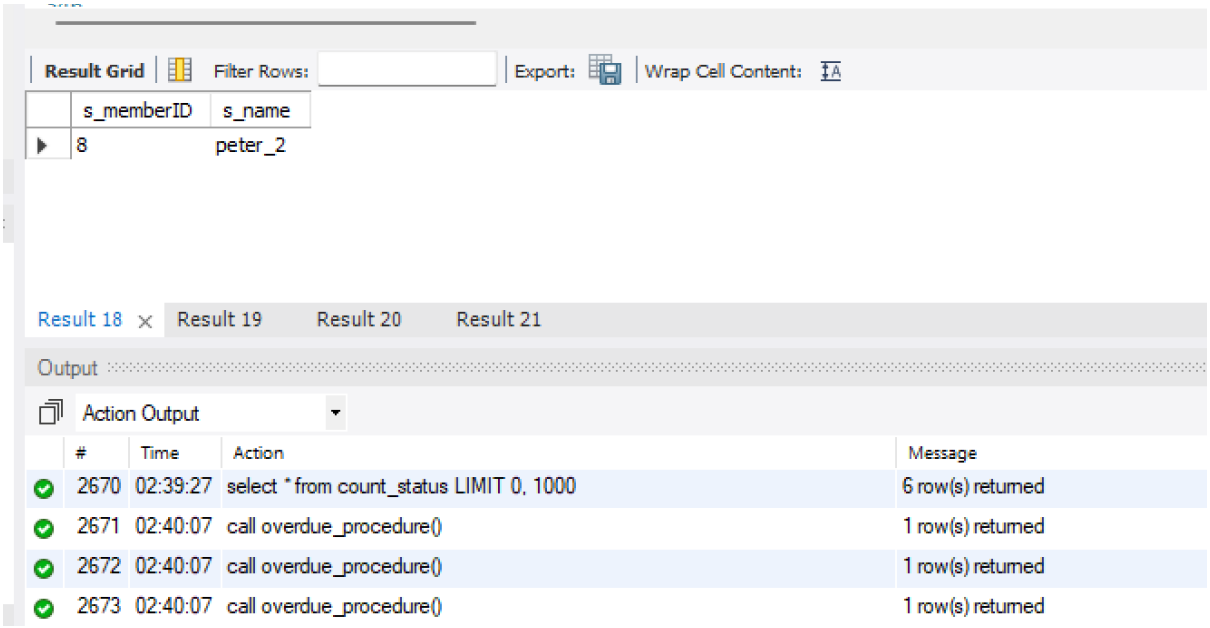
We record into our table for this recording time.

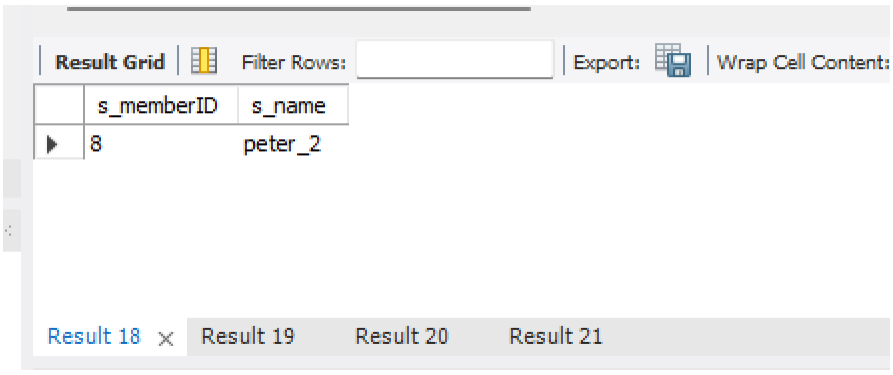
```
522 •   select * from count_status;
```

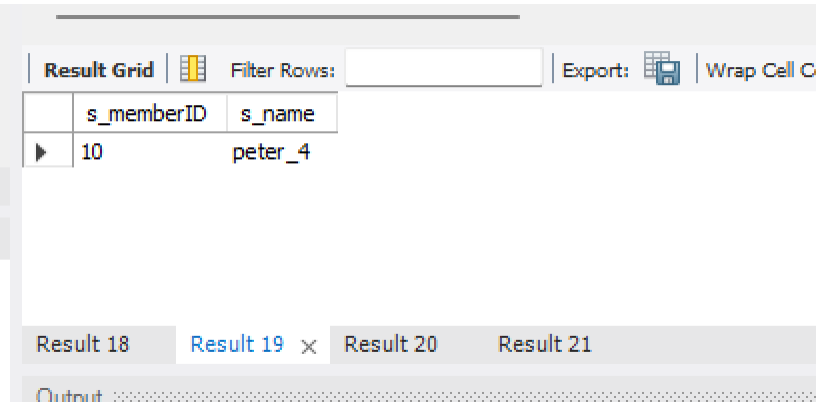
Result Grid | Filter Rows: | Export: | Wrap C

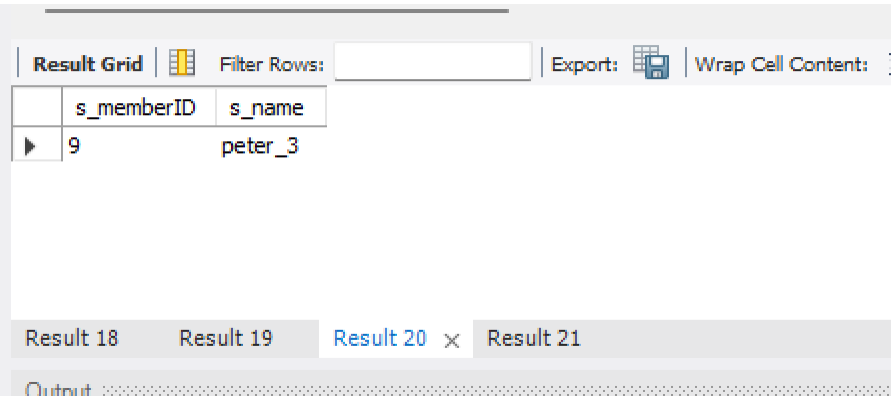
MemberID	MemberName	MemberStatus	Status_changing_date
9	NULL	SUSPENDED	2023-10-28 19:48:55
9	NULL	SUSPENDED	2023-10-28 19:49:02
9	NULL	SUSPENDED	2023-10-28 20:38:05
8	NULL	SUSPENDED	2023-10-29 01:05:59
8	NULL	SUSPENDED	2023-10-29 01:08:11

We display all results:



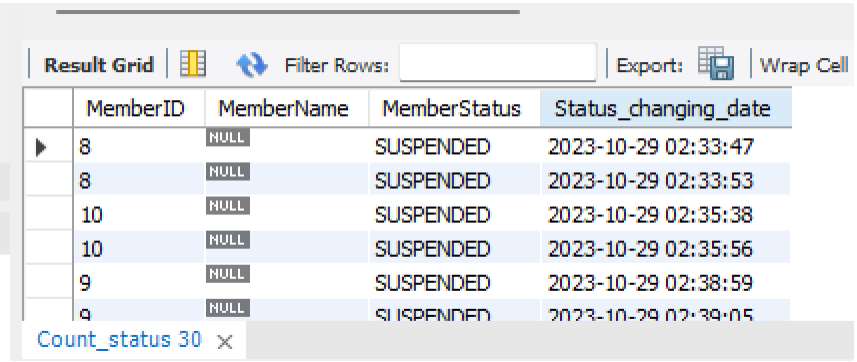






Other results 18,19,20,21 are the results that needed to be terminated as well as described in the question.

The reason is they appear twice on my table that stores the number of “suspended.



It is not hard to see why we can see that peter_2 becomes Terminated as well.

Part(2): error handler case

Next, we can test the error case:

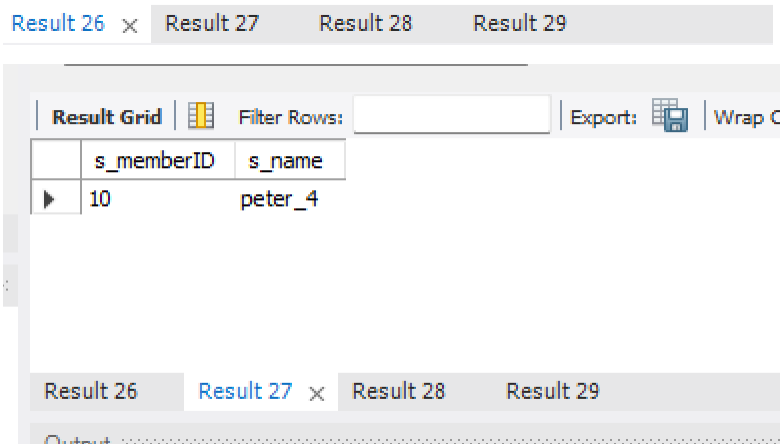
Reset Peter_3 to be regular and the fine fee=0.

Even if he has been suspended twice, but the error handle holds him back:

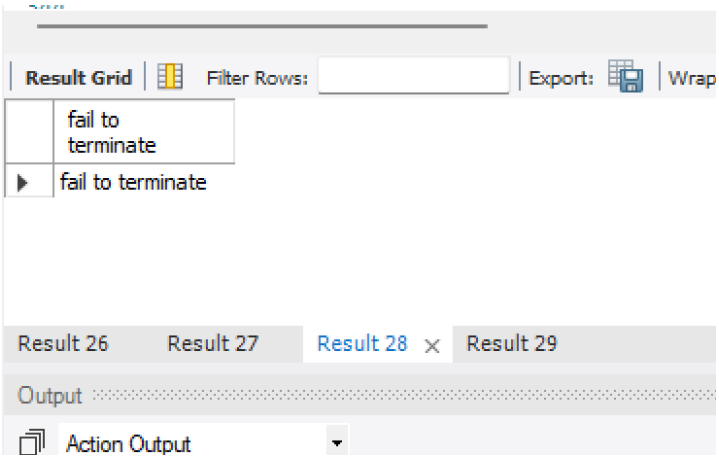
```
337 •   call overdue_procedure();
338
339 •   select MemberID,MemberStatus,fine_logging fror
340
341 •   UPDATE member
342     SET fine_logging=0
343     WHERE MemberID = 9;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

s_memberID	s_name
8	peter_2



Now we have an error handler here which blocks our selection since peter_3 has zero fine even if he has been suspended twice.



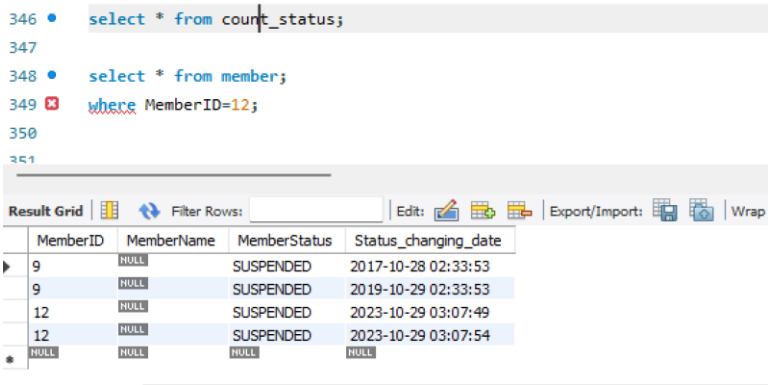
We have successfully terminated Peter_2. As we can see there are so many test cases and the only way to make sure it is correct is to set up some triggers to make constraints about it.

If we do it case by case, it would be annoying if we forget to follow some of the business rules.

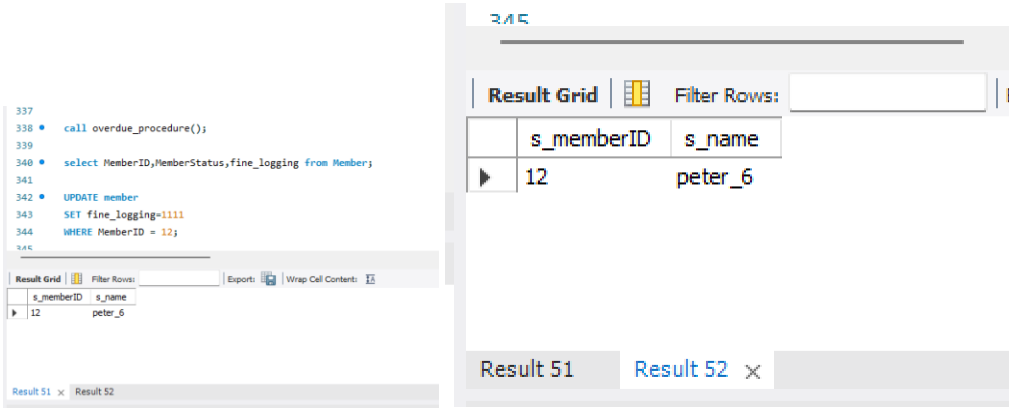
Part (3): More than 3 years range ‘s case

The only test case we haven’ t shown above.

Test case (more than 3 years range): Assume there is member number #9 has two suspended, but it happened before which is not in the range of 3 years:



However, after calling the procedure we only can display member ID 12 and it would not be turned into a terminated state since member 9 hasn't been suspended within 3 years.



We can see he is still in the state of suspension but not in termination: